CS224 Object Oriented Programming and Design Methodologies Lab Manual

Lab 10- UML Diagrams

Department of Computer Science

Dhanani School of Science and Engineering

Habib University

Fall 2025

Contents

Lab 10

UML Diagrams

10.1 Guidelines

1. Use of AI is strictly prohibited. This is not limited to the use of AI tools for code generation, debugging, or any form of assistance. If detected, it will result in immediate failure of the lab, student will be awarded 0 marks, reported to the academic integrity board and appropriate disciplinary action will be taken.
2. Absence in lab regardless of the submission status will result in 0 marks.
3. All assignments and lab work must be submitted by the specified deadline on Canvas. Late submissions will not be accepted.

10.2 Objectives

Following are the lab objectives of this lab:

1. To understand how to represent software systems using UML Class Diagrams.
2. To model real-world scenarios using object-oriented principles such as encapsulation and composition.
3. To practice identifying relationships between classes and designing appropriate data members and functions.
4. To apply UML diagram design to practical scenarios like a banking system and a graphical simulation game.

10.3 Instructions

- The diagram should be created using `https://draw.io`.
- Export the final diagram as both `.drawio` and `.png` files.
- Include the diagram in your `manual` folder.

10.4 Lab Exercises

Exercise 1 — Bank Accounts Management System

In this exercise, you are required to create a UML Class Diagram representing a small simulation of a banking system. You need to identify and model the relationships among the classes involved.

Problem Description

The banking system should model the following scenario:

- The system has a `Bank` class that maintains a list (e.g., `vector`) of `Account` objects.
- Each `Account` has attributes such as account title, account code, and balance.
- Accounts can perform two types of transactions: `Deposit` and `Withdrawal`.
- Each `Account` maintains a list of its transactions.
- A `Date` class should be created to manage all date-related tasks.
- After executing all transactions, any account with a balance less than 5000 should become *Dormant*; otherwise, it remains *Active*.

You may introduce additional classes, functions, or attributes as required to improve modularity and clarity.

Input File Processing

The program should process an input file named `bankinput.txt`. A sample is shown below:

Create Jasmin_Banda JB230 70000
Deposit GH090 30000 11-Mar
Withdrawal GH090 4000 12-April-21

The file can contain three types of entries:

1. **Create Title Code InitialDeposit:** Creates a new account with the given title, code, and initial deposit.
2. **Deposit Code Amount Date:** Deposits the specified amount into the account with the given code. The date can be written in various formats (e.g., 11-Mar or 11-March-21). The default year is 2021.
3. **Withdrawal Code Amount Date:** Withdraws the specified amount if the balance is sufficient; otherwise, the transaction fails. The date format should be flexible as above.

Output File

After processing all transactions, the program should generate an output file named `bankoutput.txt` containing information about each account. A sample is shown below:

Account Title: Saeed Ghani
Account Code: GH090
Initial Balance: 2000
Available Balance: 28000
Current Status: Active
Deposits:
1. 30000 on 11/03/21
Withdrawals:
1. 4000 on 12/04/21 Successful
2. 40000 on 15/06/21 Denied

Expected UML Classes

- `Bank`
- `Account`
- `Transaction` (Abstract or Base)
- `Deposit`
- `Withdrawal`
- `Date`

—

Exercise 2 — HU Mania

In this exercise, you are required to create a UML Class Diagram for a graphical simulation game titled **HU Mania**. The game involves animated objects (pigeon, butterfly, bee) that move across the screen with unique behaviors.

Problem Description

- Create a base class `Unit` that defines the behavior of an animated object.
  - It has a fully implemented `draw()` method to render one of its three animation frames.
  - It declares a virtual function `fly()` that will be overridden in derived classes.
- **Pigeon:** Inherits from `Unit` and overrides `fly()` to move gradually to the right while rotating.
- **Butterfly:** Inherits from `Unit` and overrides `fly()` to move diagonally (right-down and right-up alternately) across the screen.
- **Bee:** Inherits from `Unit` and overrides `fly()` to move only to the right, occasionally hovering (1% probability each frame for 10 frames). When it reaches the right edge, it should be removed from the vector.
- All objects should animate using three frames stored in the `assets` folder.
- Create an `ObjectCreator` class with a function `Unit* getObject()` that randomly creates one of the three objects (`Pigeon`, `Butterfly`, `Bee`) and returns its pointer.
- Maintain a single list (e.g., `vector<Unit*>`) that holds all active objects.
- Iterate through this list to call `draw()` and `fly()` for each object.
- Delete dynamically allocated objects when the game ends or when a bee exits the screen.
- Identify object types when removing bees (use RTTI or type checking).

Expected UML Classes

- `Unit`
- `Pigeon`
- `Butterfly`
- `Bee`
- `ObjectCreator`

—

Exercise 3 — Project UML Diagram

In this exercise, you are required to brainstorm with your group members and create a UML Class Diagram for your project. You should identify all relevant classes, their relationships, and major functions. Use the guidelines from the above two exercises to ensure clarity and consistency.